

Design and Analysis of Algorithms

- *Course Code:* BCSE204L
- *Course Type:* Theory (ETH)
- *Slot:* A1+TA1 & & A2+TA2
- *Class ID:* VL2023240500901
VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 3

Module:3	String Matching Algorithms
----------	----------------------------

5 hours

Naïve String-matching Algorithms,
KMP algorithm,
Rabin-Karp Algorithm,
Suffix Trees.

String Matching Algorithm

- **Definition:** A String Matching Algorithm is a method used to find occurrences of a particular pattern (referred to as the "needle" or "**substring**") within a larger text (referred to as the "**haystack**" or "**string**").
- **Input:** The main input consists of:
 - A text or string (the haystack) in which we want to search for occurrences of another string.
 - A pattern or substring (the needle) that we want to find within the haystack.
- The **output** typically includes:
 - The positions or indices where the pattern occurs within the text.
 - The count of occurrences.
 - In some cases, additional information might be provided, such as the length of the longest common substring, etc.

String Matching Algorithm: Application

These algorithms are widely used in various applications such as text processing, data mining, bioinformatics, and more.

- **Text Search Engines:** Search engines like Google, Bing, and others use string matching algorithms to efficiently search through vast amounts of textual data and retrieve relevant information.
- **Data Compression:** String matching algorithms are used in data compression techniques such as LZ77 and LZ78, which are fundamental to various compression algorithms like ZIP.
- **DNA Sequencing:** In bioinformatics, string matching algorithms are used for DNA sequence alignment, where researchers compare sequences to identify similarities and differences, aiding in genomic analysis.
- **Network Intrusion Detection:** String matching algorithms are used in intrusion detection systems to identify patterns in network traffic that could indicate malicious activity.
- **Spell Checkers:** String matching algorithms are utilized in spell checkers to suggest corrections or alternatives for misspelled words by comparing them with a dictionary of correctly spelled words.
- **Compiler Design:** Compilers often use string matching algorithms in lexical analysis to tokenize source code, identifying keywords, identifiers, and symbols.
- **Natural Language Processing (NLP):** In tasks like named entity recognition or sentiment analysis, string matching algorithms can be used to identify specific phrases or patterns within text data.
- **Fraud Detection:** String matching algorithms can help detect fraudulent activities by searching for suspicious patterns in large datasets, such as financial transactions.

String Matching Algorithm

- The Naïve String-Matching Algorithm, also known as the Brute-Force or Simple Matching Algorithm, is one of the simplest algorithms for finding occurrences of a pattern within a text.
- It works by systematically comparing the pattern with substrings of the text.

```
NaiveStringMatch(text, pattern):
```

```
    n = length of text
```

```
    m = length of pattern
```

```
    occurrences = []
```

```
    for i from 0 to n - m do:
```

```
        j = 0
```

```
        while j < m and text[i+j] = pattern[j]:
```

```
            j = j + 1
```

```
        if j = m:
```

```
            add i to occurrences
```

```
    return occurrences
```

String:

A B C D E F G H

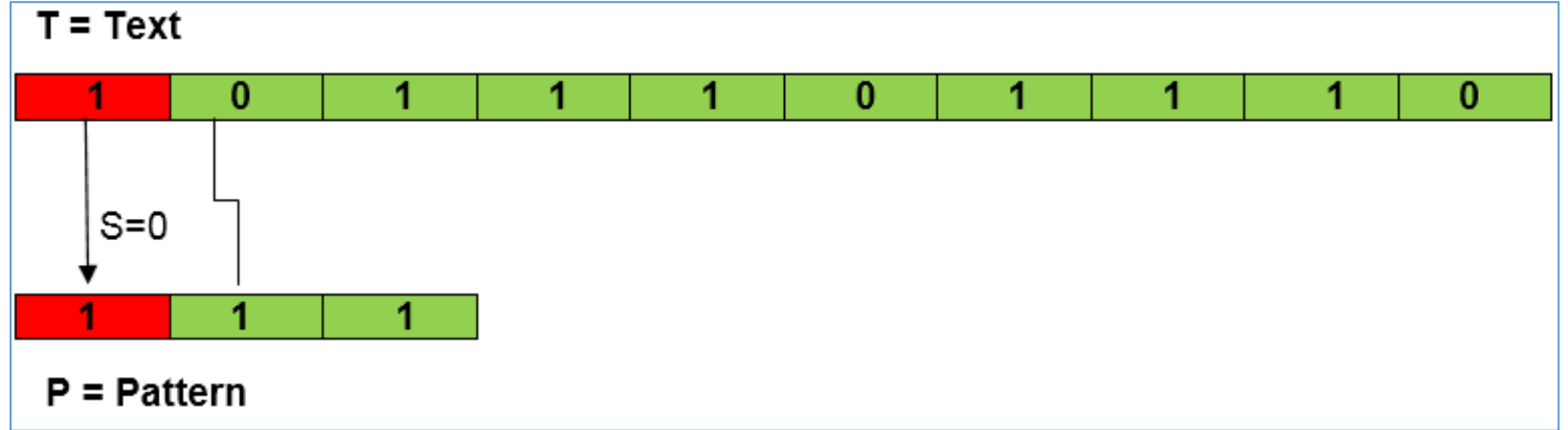
Pattern:

SCALER
Topics

String Matching Algorithm: Example

T = 1011101110

P = 111



NaiveStringMatch(text, pattern):

n = length of text

m = length of pattern

occurrences = []

for i from 0 to n - m do:

j = 0

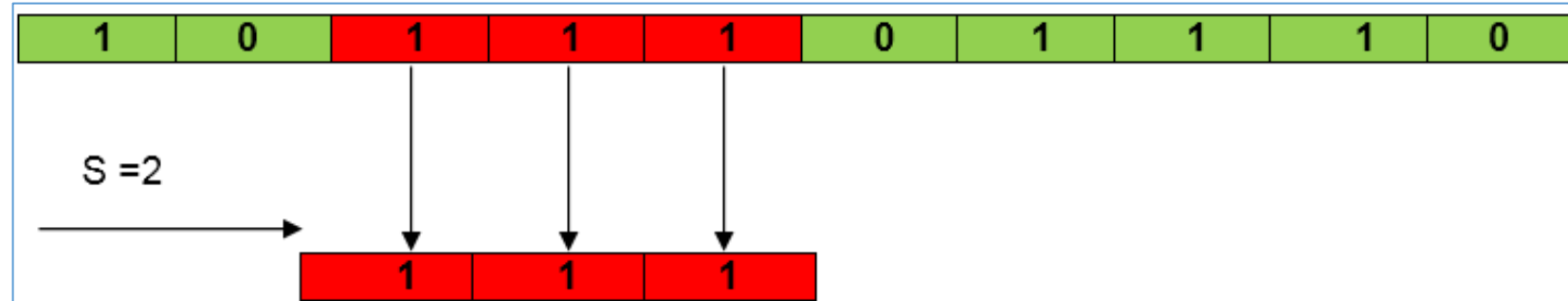
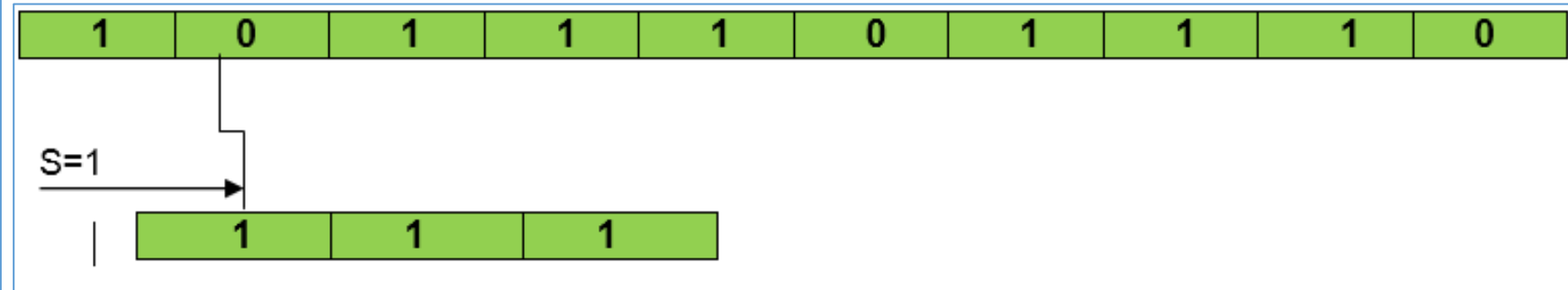
while j < m and text[i+j] = pattern[j]:

j = j + 1

if j = m:

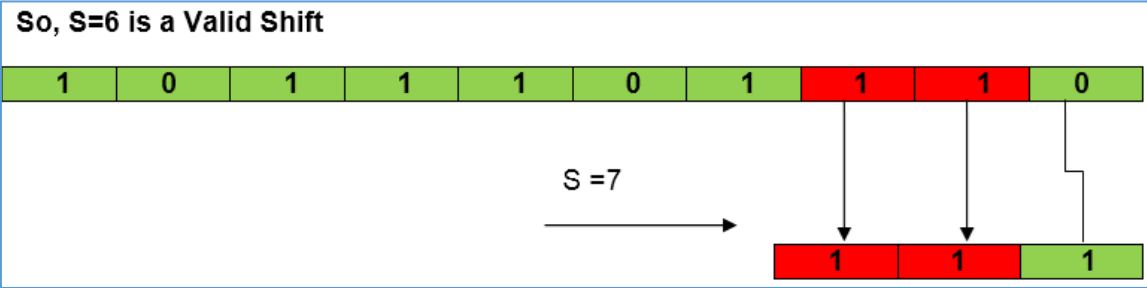
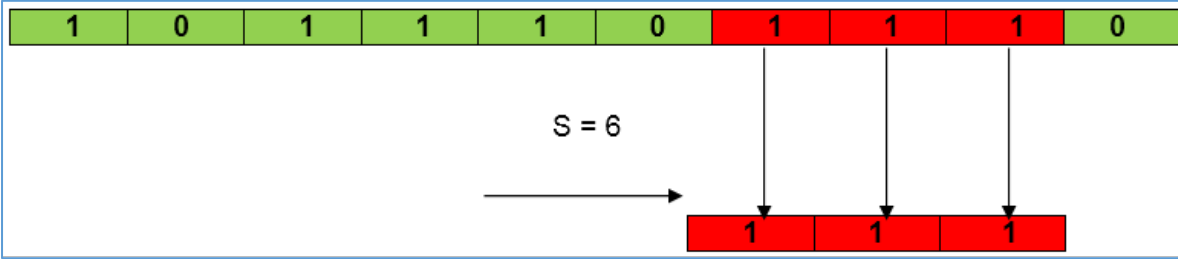
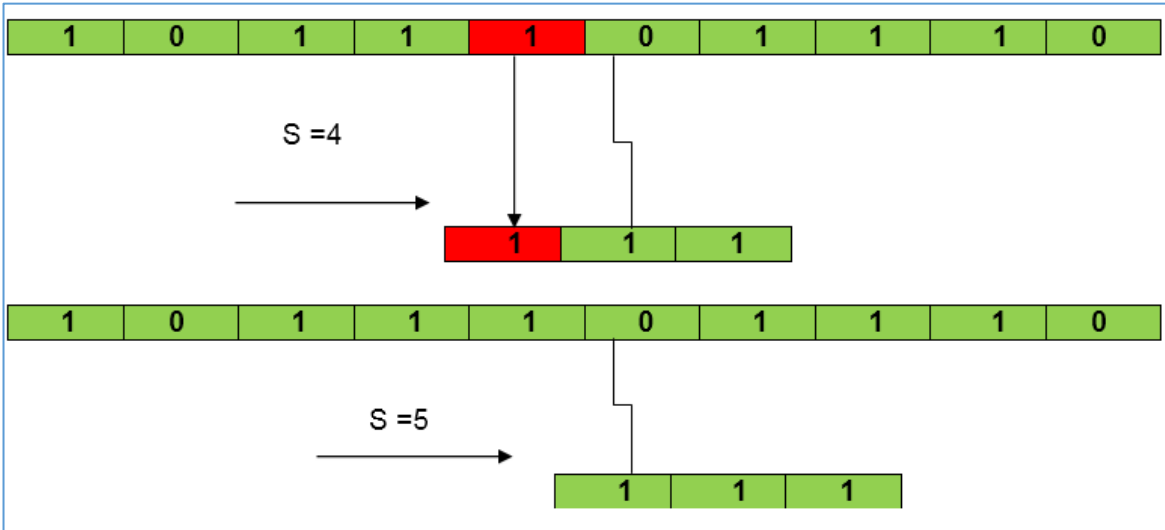
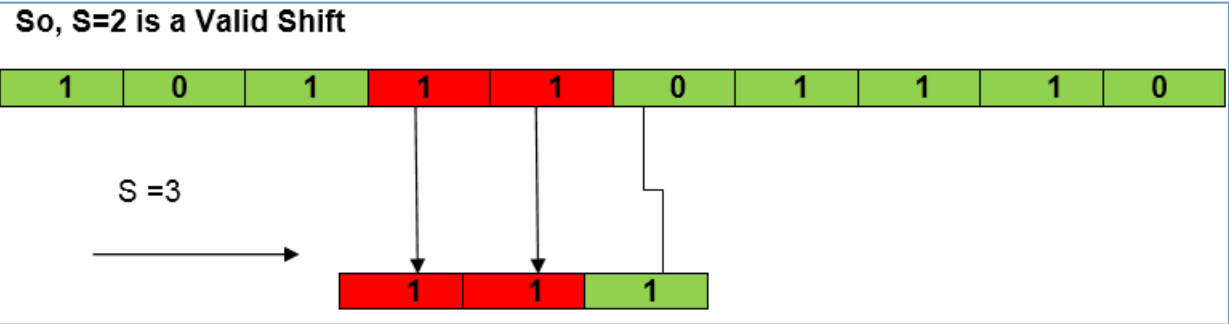
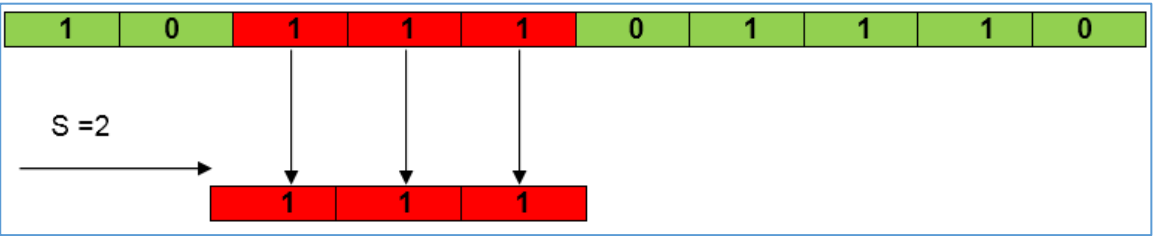
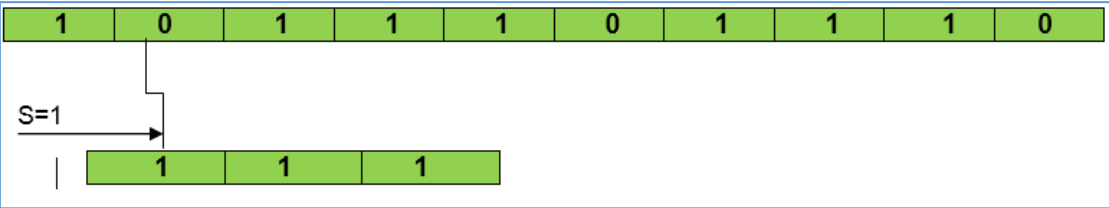
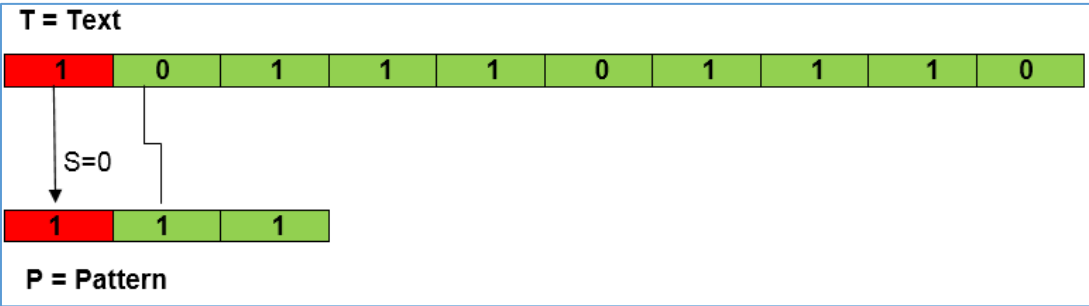
add i to occurrences

return occurrences



String Matching Algorithm: Example

T = 1011101110
P = 111



String Matching Algorithm

NaiveStringMatch(text, pattern):

n = length of text

m = length of pattern

occurrences = []

for i from 0 to n - m do:

 j = 0

 while j < m and text[i+j] = pattern[j]:

 j = j + 1

 if j = m:

 add i to occurrences

return occurrences

Time Complexity:

- The outer loop iterates $n - m + 1$ times.
- In the worst case, the inner while loop iterates m times for each iteration of the outer loop.
- Therefore, the time complexity is approximately $O((n - m + 1) * m)$, which simplifies to $O(n * m)$.

Note: The Naïve String-Matching Algorithm is simple and easy to understand but can be inefficient for large texts or patterns due to its $O(n * m)$ time complexity.

It's mainly used for educational purposes or for **small-scale problems** where efficiency is not critical.

For larger-scale string matching tasks, more sophisticated algorithms like **Knuth-Morris-Pratt (KMP)** or **Boyer-Moore, Rabin-Karp Algorithm, Suffix Trees** are typically preferred.

Example: The text and pattern is given below and find the index value at which pattern matched with text.

text : a b c d e a b f g f
pattern : e a b

Step 1 : a b c d e a b f g h
 e a b ...pattern is not matched.

Step 2 : a b c d e a b f g h
 e a b ...pattern is not matched.

Step 3 : a b c d e a b f g h
 e a b ...pattern is not matched.

Step 4 : a b c d e a b f g h
 e a b ...pattern is not matched.

Step 4 : a b c d e a b f g h
 e a b ...pattern is matched.

Practice:

T=abcbacbaac

P=cba

Find all the Valid Shift

Pattern found at the following positions:

Start index: 2, End index: 4

Start index: 5, End index: 7

Any

Question



PresenterMedia

Thank You!

**FOR YOUR
ATTENTION**

